Mind map 1/2 — comment marche une itération

(Nk) = équation k de `sequential_cokriging.pdf`
démonstrations dans `complements_mathematiques.pdf`

Entrées
- plan imbriqué évalué $\mathcal{D}_L \subset \cdots \subset \mathcal{D}_1$ (`ExperimentData`)
- coûts $c_1 < \cdots < c_L$ (`data.costs`), domaine et `feasibility`
- points d'intégration : Sobol (global) ou états VPP (orienté objectif)
- simulateur (`BaseSimulator`)

1 Ajuster le modèle

Question : que sait-on ?
- `model = CoKriging(L)` ; `model.fit(data)`
- un GP δ_j par niveau, ajusté sur ses résidus $d_j = z_j - \rho_{j-1} z_{j-1}$
- ρ appris ⇒ gains $P_j = \rho_j \cdots \rho_{L-1}$
- hyperparamètres **figés** jusqu'à la fin de l'itération

2 Décomposer l'incertitude

Question : où, et à quel niveau, ignore-t-on ?
- $\mathrm{Var}[Z_L(\boldsymbol{x})] = \sum_j P_j^2 \sigma_j^2(\boldsymbol{x})$ (N2)
  part de chaque niveau dans l'incertitude HF
- erreurs LOO de chaque δ_j : $z_{j,i}^2 = \alpha_i^2 / [\widetilde{\boldsymbol{K}}^{-1}]_{ii}$ (N3)
- $z^2 \gg 1$: le modèle se trompe plus qu'il ne le croit

3 Où calculer ? → $\boldsymbol{x}_0$

Question : où le modèle est-il le moins fiable ?
- variance ajustée (N4)
  $k_{\mathrm{adj}}(\boldsymbol{x}) = \sum_j P_j^2 \sigma_j^2(\boldsymbol{x}) \, (1 + z_{j,\mathrm{voisin}}^2)$
- voisin = point le plus corrélé (métrique du noyau)
- $\boldsymbol{x}_0 = \arg\max k_{\mathrm{adj}}$: évolution différentielle, dans le domaine réalisable (`feasibility`)

4 Jusqu'à quel niveau ? → $l^\star$

Question : quel calcul rapporte le plus par seconde CPU ?
- gain exact si $\boldsymbol{x}_0$ est calculé aux niveaux $1..l$: (N5)
  $G_l = \sum_{j \le l} P_j^2 \int \frac{k_j^n(\boldsymbol{x}, \boldsymbol{x}_0)^2}{\sigma_j^2(\boldsymbol{x}_0) + \tau_j^2} d\boldsymbol{x}$
- intégrale = moyenne sur les points d'intégration
- coût cumulé $C_l = c_1 + \cdots + c_l$ (imbrication)
- $l^\star = \arg\max_l G_l / C_l$ (N6)
  ρ^2 **inclus** : correction de LG

5 Calculer

Action : lancer les codes
- niveaux $1, 2, \ldots, l^\star$ en $\boldsymbol{x}_0$, du moins cher au plus cher
- `simulator.evaluate(x0, j)` puis `data.add_observation(j, x0, y)`
- échec (NaN) ⇒ arrêt : les niveaux au-dessus ne sont pas lancés
- le plan reste **imbriqué**

6 Arrêter ?

Question : est-ce assez précis ?
- $Q^2_{\mathrm{LOO}} \ge$ cible (`leave_one_out` + `prediction_scores`)
- ou budget de calcul épuisé
- plus tard, avec la VPP : incertitude sur la vitesse $\sigma_V <$ tolérance

Une itération du co-krigeage séquentiel

But : rendre le niveau haut précis au moindre coût de calcul

$\boldsymbol{x}_0$

$\boldsymbol{x}_0,\ l^\star$

non : itération suivante

oui

Sorties
- surrogate final `CoKriging` (+ `to_dict`)
- données enrichies, toujours imbriquées
- historique : coût, Q^2, niveaux choisis

Légende
- fait et testé
- à faire maintenant, dans l'ordre
- plus tard (projet VPP)

mfego n'est pas modifié : on appelle `src/cokriging.py`

Construire `vpp_acceleration/src/sequential_design.py`

dans l'ordre : chaque étape testée avant la suivante

1 Noyau de calcul

État : écrit et testé (hors dépôt)
- `level_weights`, `level_variances`, `posterior_cov` : (N2)
- `loo_z2`, `adjustment`, `adjusted_variance` : (N3)–(N4)
- `delta_imse`, `choose_level` : (N5)–(N6)

Test : décomposition à 10^{-4}, LOO à $\leq 10^{-2}$, gain contre reconditionnement à $\leq 2 \cdot 10^{-3}$

2 Choix du point

Étape (3) de la page 1
- enveloppe vectorisée de `differential_evolution`, reprise d'`EGOOptimizer._find_next_point`
- masque `feasibility` : critère nul hors du domaine

Test : en 1D, $\boldsymbol{x}_0$ coïncide avec le maximum de k_{adj} sur une grille fine

3 Boucle ask / tell / run

Étapes (1) à (6) de la page 1
- classe `SequentialCoKriging(data, simulator, ...)`
- `tell` : niveaux $1..l^\star$ dans l'ordre, arrêt au premier NaN
- historique (coût, Q^2, niveaux), critère d'arrêt

Test : `missing_points_for_nesting` vide après chaque itération

4 Plan initial sous contraintes

Ce qui manque dans mfego
- LHS sur-échantillonné → filtre `feasibility` → maximin → sous-ensembles imbriqués
- `generate_nested_design` n'accepte pas de contraintes aujourd'hui

Test : tous les points réalisables, plan imbriqué

5 Valider l'algorithme

Sur des problèmes connus, avant la VPP
- Forrester, Hartmann, profil (`benchmarks/cokriging`)
- contre un LHS imbriqué **à coût égal**, 10 graines
- variantes : règle de LG ou corrigée ; avec ou sans ajustement LOO

Test : RMSE plus faible à coût égal, couverture à 95 % conservée

6 Généraliser pour la VPP

Mêmes formules, autres arguments
- points d'intégration en argument : Sobol ou états d'équilibre VPP
- plusieurs sorties : un `CoKriging` par composante du torseur, poids $w_o = (\partial V / \partial F_o)^2$
- diagnostic N_{eff} en grande dimension ; option sans ajustement ($a_j = 1$)

Test : un poids unique redonne le cas mono-sortie

7 Brancher sur le projet

Plus tard, après validation
- 0 cadrage → 1 chaîne de solveurs → 2 surrogate de base → 3 VPP → 4 raffinement orienté objectif (cet algorithme) → 5 validation CFD